

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по ИДЗ
по дисциплине «Математическая логика»
Тема: использование бинарных решающих диаграмм для решения
логических задач

Студент гр. 3343

Жучков О. Д.

Преподаватель

Шошмина И. В.

Санкт-Петербург

2025

Задание.

Решить модифицированную задачу Эйнштейна с помощью бинарной решающей диаграммы программным образом (C++ с библиотекой BuDDy). Задача определяется следующими параметрами.

- Имеется $N = 9$ объектов с $M = 4$ свойствами. Свойства принимают N различных значений.
- Имеются индивидуальные соседские отношения, склейка, обязательное количество ограничений n_1, n_2, n_3, n_4

Уровень выполнения ИДЗ: продвинутый — выполняется вариант без склейки и со склейкой.

Данные варианта: $v_1 = 18, v_2 = 1, v_3 = 3: n_1 = 7, n_2 = 5, n_3 = 5, n_4 = 4$

		*	
18		0	
		*	

Рисунок 1 – Шаблон соседских отношений варианта.

Выполнение работы.

Написаны функции для определения номера соседа объекта по шаблону: `get_neighbor_left` и `get_neighbor_right`. Объекты нумерованы от 0 до $N-1$ и расположены по порядку на поле, слева-направо сверху-вниз. Для нахождения номера соседа необходимо прибавить (отнять) ширину поля, например для объекта 3 соседними являются 0 и 6. Если нет склейки и у объекта нельзя определить соседний, то возвращается N - несуществующий объект.

```

layout:
012
345
678
generalized for n=SQRT_N:
012.....n-1
n.....2*n-1
.....
n*(n-1)..n^2-1
*/
unsigned get_neighbor_left(unsigned i, bool loop=false){
    if (i - SQRT_N > N)
        if (loop)
            return N - SQRT_N + i;
        else
            return N; // coordinate N means undefined
    else
        return i - SQRT_N;
}

```

Рисунок 2 – Функция для нахождения соседнего объекта.

Функция `make_p_funcs()` создает массив BDD объектов для $p(k,i,j)$, принимающей истинное значение, если свойство k объекта i имеет значение j . Данный массив используется для всех последующих ограничений задачи. Шесть функций типа `restrict_type_X()` описывают все виды ограничений в задаче Эйнштейна, принимают на вход BDD объект задачи и возвращает его после применения к нему бинарной решающей диаграммы ограничения.

1. Принимает на вход k, i, j — конкретное свойство объекта имеет конкретное значение
2. Принимает на вход k_1, j_1, k_2, j_2 — соответствие между значениями двух разных свойств одного объекта.
3. Принимает k_1, j_1, k_2, j_2 , одно из двух направлений — объект с первым свойством сосед по направлению объекту со вторым свойством.
4. Принимает k_1, j_1, k_2, j_2 — расположение рядом: дизъюнкция двух ограничений 3.
5. Обязательное ограничение для задачи Эйнштейна — объекты не могут иметь одинаковые свойства.

6. Ограничение количества значений свойств, значения не могут выходить за границу N.

Программа масштабируема, возможно изменение количества объектов N и количества свойств M.

Выбраны ограничения:

```
g = restrict_type_1(g, 0, 0, 0);  
g = restrict_type_1(g, 1, 1, 1);  
g = restrict_type_1(g, 2, 2, 2);  
g = restrict_type_1(g, 3, 3, 3);  
g = restrict_type_1(g, 0, 4, 4);  
g = restrict_type_1(g, 1, 5, 5);  
g = restrict_type_1(g, 2, 6, 6);  
  
g = restrict_type_1(g, 1, 8, 0);  
g = restrict_type_1(g, 0, 2, 1);
```

Рисунок 3 – Ограничения первого типа.

```
g = restrict_type_2(g, 0, 8, 3, 8);  
g = restrict_type_2(g, 1, 7, 2, 7);  
g = restrict_type_2(g, 2, 6, 1, 6);  
g = restrict_type_2(g, 3, 5, 0, 5);  
g = restrict_type_2(g, 0, 4, 3, 4);
```

Рисунок 4 – Ограничения второго типа.

```

g = restrict_type_3(g, 0, 5, 1, 3, false);
g = restrict_type_3(g, 3, 7, 2, 3, false);
g = restrict_type_3(g, 1, 1, 1, 4);
g = restrict_type_3(g, 2, 4, 3, 0);
g = restrict_type_3(g, 3, 2, 0, 3);

```

Рисунок 5 – Ограничения третьего типа.

```

g = restrict_type_4(g, 1, 6, 0, 3);
g = restrict_type_4(g, 2, 1, 2, 5);
g = restrict_type_4(g, 0, 6, 3, 7);
g = restrict_type_4(g, 3, 6, 1, 5);

```

Рисунок 6 – Ограничения четвертого типа.

При наборе ограничений остается всего 16 возможных решений:

458 nodes
16 solutions:

0:	0	8	1	2
1:	8	1	0	8
2:	1	2	2	6
3:	3	3	5	3
4:	4	4	4	4
5:	6	5	3	1
6:	5	6	6	5
7:	2	7	7	0
8:	7	0	8	7

Рисунок 7 – Решение задачи.

Пример физической интерпретации задачи:

9 людей разной национальности живут в 9 разных домах разного цвета, с разным количеством этажей и разным типом окон.

1. Первый дом белый
2. Второй дом двухэтажный
3. Третий дом имеет треугольные окна
4. В четвертом доме живет поляк
5. Пятый дом зеленый
6. Шестой дом шестиэтажный
7. Седьмой дом имеет семиугольные окна
8. Девятый дом одноэтажный
9. Третий дом красный
10. Швед живет в черном доме
11. Фиолетовый дом имеет восьмиугольные окна
12. Семиэтажный дом имеет семиугольные окна
13. Англичанин живет в голубом доме
14. Норвежец живет в зеленом доме
15. Голубой дом справа от четырехэтажного дома
16. Немец живет в доме справа от дома с квадратными окнами
17. Двухэтажный дом слева от пятиэтажного
18. Дом с пятиугольными окнами слева от дома, где живет русский
19. Американец живет слева от желтого дома
20. Семиэтажный дом рядом с желтым домом
21. Дом с круглыми окнами рядом с домом с шестиугольными окнами
22. Синий дом рядом с домом немца
23. Датчанин живет рядом с шестиэтажным домом

Одно из решений (рис. 7):

1. Первый дом белый, девятиэтажный, с круглыми окнами, в нем живет американец

2. Второй дом черный, двухэтажный, без окон, в нем живет швед
3. Третий дом красный, трехэтажный, с треугольными окнами, в нем живет датчанин
4. Четвертый дом желтый, четырехэтажный, с шестиугольными окнами, в нем живет поляк
5. Пятый дом зеленый, пятиэтажный, с пятиугольными окнами, в нем живет норвежец
6. Шестой дом синий, шестиэтажный, с квадратными окнами, в нем живет украинец
7. Седьмой дом голубой, семиэтажный, с семиугольными окнами, в нем живет англичанин
8. Восьмой дом оранжевый, восьмиэтажный, с восьмиугольными окнами, в нем живет русский
9. Девятый дом фиолетовый, одноэтажный, с девятиугольными окнами, в нем живет немец

В функциях для нахождения соседа добавлена обработка склейки. По числам варианта $v2 = 1$ — горизонтальная склейка, однако с данным шаблоном соседских отношений при горизонтальной склейке ничего не изменится, поэтому для демонстрации работы используется вертикальная склейка (верхней и нижней границ).

26983 nodes
1424 solutions:

Рисунок 8 – Число узлов и решений задачи.

При запуске программы с включенной склейкой и теми же ограничениями увеличивается количество решений почти в 100 раз. Добавим ограничения:

1. В восьмом доме живет украинец
2. В фиолетовом доме живет русский
3. Красный дом слева от черного
4. Оранжевый дом рядом с фиолетовым

405 nodes
8 solutions:

0:	0	7	7	2
1:	7	1	1	0
2:	1	8	2	7
3:	3	3	0	3
4:	4	4	5	4
5:	8	5	8	8
6:	5	6	6	5
7:	2	2	4	1
8:	6	0	3	6

Рисунок 9 – Решение задачи.

1. Первый дом белый, восьмиэтажный, с восьмиугольными окнами, в нем живет американец
2. Второй дом фиолетовый, двухэтажный, с круглыми окнами, в нем живет русский
3. Третий дом красный, девятиэтажный, с треугольными окнами, в нем живет немец
4. Четвертый дом желтый, четырехэтажный, без окон, в нем живет поляк
5. Пятый дом зеленый, пятиэтажный, с шестиугольными окнами, в нем живет норвежец

6. Шестой дом черный шестиэтажный, с девятиугольными окнами, в нем живет швед
7. Седьмой дом голубой, семиэтажный, с шестиугольными окнами, в нем живет англичанин
8. Восьмой дом оранжевый, трехэтажный, с пятиугольными окнами, в нем живет украинец
9. Девятый дом синий, одноэтажный, с четырехугольными окнами, в нем живет датчанин

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД

main.cpp:

```
#include <bdd.h>
#include <fstream>

# define A 6
# define B 3343
# define C 18
# define V1 ((A+B) % 28) + 1
# define V2 ((A+C) % 2) + 1
# define V3 (A % 4) + 1

# define N 9 // число объектов (и значений свойства)
# define SQRT_N 3 // с округлением вверх (для расположения на
поле)
# define M 4 // число свойств
# define LOG_N 4
# define POW2_N 16
# define N_TRAIT_VAR N * M * LOG_N // число булевых переменных
для описания свойств объектов
# define LOOP false

using namespace std;

ofstream out;

bdd p[M][N][POW2_N];
char var [ N_TRAIT_VAR ];

/*
v1 = 18:
. * . - "left"
. 0 .
. * . - "right"
v2 = 1 - horizontal loop
however, does not change solution for the neighbor relationship
described above
using v2 = 2 - vertical loop to showcase the algorithm working
layout:
012
345
678
generalized for n=SQRT_N:
012.....n-1
n.....2*n-1
.....
n*(n-1)..n^2-1
*/
unsigned get_neighbor_left(unsigned i, bool loop=false){
    if (i - SQRT_N > N)
```

```

        if (loop)
            return N - SQRT_N + i;
        else
            return N; // coordinate N means undefined
    else
        return i - SQRT_N;
}

unsigned get_neighbor_right(unsigned i, bool loop=false){
    if (i + SQRT_N > N - 1)
        if (loop)
            return i + SQRT_N - N;
        else
            return N;
    else
        return i + SQRT_N;
}

void print (void){
    for ( unsigned i = 0; i < N ; i ++){
        out << i << ": ";
        for ( unsigned j = 0; j < M ; j ++){
            unsigned J = i* M * LOG_N + j * LOG_N ;
            unsigned num = 0;
            for ( unsigned k = 0; k < LOG_N ; k ++ ) num +=
( unsigned ) (var [ J + k ] << k ) ;
            out << num << ' ';
        }
        out << endl;
    }
    out << endl;
}

void build (char * varset, unsigned n, unsigned I){
    if (I == n - 1){
        if (varset[I] >= 0){
            var[I] = varset[I];
            print();
            return;
        }
        var[I] = 0;
        print();
        var[I] = 1;
        print();
        return;
    }
    if (varset[I] >= 0){
        var[I] = varset[I];
        build(varset, n, I + 1);
        return;
    }
    var[I] = 0;
    build(varset, n, I + 1);
}

```

```

    var[I] = 1;
    build(varset ,n , I + 1);
}

void fun(char * varset, int size){
    build(varset, size, 0);
}

void make_p_funcs(){
    for (unsigned k = 0; k < M; k++){
        for (unsigned i = 0; i < N; i++){
            for (unsigned j = 0; j < POW2_N; j++){
                p[k][i][j] = bddtrue;
                for (unsigned bit = 0; bit < LOG_N; bit++){
                    p[k][i][j] &= ((j >> bit) & 1)
                        ? bdd_ithvar(i*M*LOG_N + k*LOG_N + bit) :
bdd_nithvar(i*M*LOG_N + k*LOG_N + bit);
                }
            }
        }
    }

// n1 = 7
bdd restrict_type_1(bdd F, unsigned k, unsigned i, unsigned j){
    return F & p[k][i][j];
}

// n2 = 5
bdd restrict_type_2(bdd F, unsigned k1, unsigned j1, unsigned k2,
unsigned j2){
    for (unsigned i = 0; i < N; i++){
        F &= (p[k1][i][j1] & p[k2][i][j2]) | (!p[k1][i][j1] & !
p[k2][i][j2]); // p[k1,i,j1] <=> p[k2,i,j2]
    }
    return F;
}

// n3 = 5
bdd restrict_type_3(bdd F, unsigned k1, unsigned j1, unsigned k2,
unsigned j2, bool left=true){
    for (unsigned i = 0; i < N; i++){
        unsigned neighbor_i = (left) ? get_neighbor_left(i,
LOOP) : get_neighbor_right(i, LOOP);
        if (neighbor_i == N){
            F &= !p[k2][i][j2] & !p[k1][(left) ?
get_neighbor_left(i, true) : get_neighbor_right(i, true)][j1];
        }
        else
            F &= (p[k1][neighbor_i][j1] & p[k2][i][j2])
                | (!p[k1][neighbor_i][j1] & !p[k2][i][j2]); //
p[k1,neighbor,j1] <=> p[k2,i,j2]
    }
    return F;
}

```

```

// n4 = 4
bdd restrict_type_4(bdd F, unsigned k1, unsigned j1, unsigned k2,
unsigned j2){
    F &= restrict_type_3(bddtrue, k1, j1, k2, j2, true) |
restrict_type_3(bddtrue, k1, j1, k2, j2, false);
    return F;
}

bdd restrict_type_5(bdd F){
    for (unsigned i1 = 0; i1 < N; i1++){
        for (unsigned i2 = i1+1; i2 < N; i2++){
            //cout << i1 << " " << i2 << endl;
            bdd temp = bddtrue;
            if (i1 != i2)
                for (unsigned k = 0; k < M; k++){
                    for (unsigned j = 0; j < N; j++){
                        temp &= bdd_imp(p[k][i1][j], !p[k][i2]
[j]); //no two objects have equal traits
                    }
                }
            F &= temp;
        }
    }
    return F;
}

bdd restrict_type_6(bdd F){
    for (unsigned i = 0; i < N; i++){
        for (unsigned k = 0; k < M; k++){
            bdd temp = bddtrue;
            for (unsigned j = N; j < POW2_N; j++){
                temp &= !p[k][i][j]; //trait can't be a value
higher than N
            }
            F &= temp;
        }
    }
    return F;
}

int main (void) {
    cout << "Данные варианта: v1 - " << V1 << "; v2 - " << V2 <<
"; v3 - " << V3 << endl;
    bdd_init(1000000000, 1000000);
    bdd_setcacheratio(10);
    bdd_setvarnum(N_TRAIT_VAR);
    cout << "init done" << endl;
    make_p_funcs();
    cout << "p_func_done" << endl;
    bdd F = bddtrue;

    F = restrict_type_6(F);
    cout << "restrict_6" << endl;

    F = restrict_type_1(F, 0, 0, 0);
    F = restrict_type_1(F, 1, 1, 1);
    F = restrict_type_1(F, 2, 2, 2);
    F = restrict_type_1(F, 3, 3, 3);
}

```

```

F = restrict_type_1(F, 0, 4, 4);
F = restrict_type_1(F, 1, 5, 5);
F = restrict_type_1(F, 2, 6, 6);

F = restrict_type_1(F, 1, 8, 0);
F = restrict_type_1(F, 0, 2, 1);

//g = restrict_type_1(g, 3, 7, 1); If loop
cout << "restrict 1" << endl;

F = restrict_type_2(F, 0, 8, 3, 8);
F = restrict_type_2(F, 1, 7, 2, 7);
F = restrict_type_2(F, 2, 6, 1, 6);
F = restrict_type_2(F, 3, 5, 0, 5);
F = restrict_type_2(F, 0, 4, 3, 4);

//g = restrict_type_2(g, 0, 7, 3, 0);
cout << "restrict 2" << endl;

F = restrict_type_3(F, 0, 5, 1, 3, false);
F = restrict_type_3(F, 3, 7, 2, 3, false);
F = restrict_type_3(F, 1, 1, 1, 4);
F = restrict_type_3(F, 2, 4, 3, 0);
F = restrict_type_3(F, 3, 2, 0, 3);

//g = restrict_type_3(g, 0, 1, 0, 8);
cout << "restrict_3" << endl;

F = restrict_type_4(F, 1, 6, 0, 3);
F = restrict_type_4(F, 2, 1, 2, 5);
F = restrict_type_4(F, 0, 6, 3, 7);
F = restrict_type_4(F, 3, 6, 1, 5);

//g = restrict_type_4(g, 0, 2, 0, 7);
cout << "restrict_4" << endl;

F = restrict_type_5(F);
cout << "restrict_5" << endl;

out.open ("out_loop.txt");
out << bdd_nodecount(F) << " nodes" << endl;
unsigned long long satcount = (unsigned long
long)bdd_satcount(F);
cout << satcount << " solutions\n" << endl;
out << satcount << " solutions:\n" << endl;
if (satcount > 0) bdd_allsat(F, fun);
out.close();
return 0;
}

```